UNIVERSIDAD VERACRUZANA

FACULTAD DE ESTADÍSTICA E INFORMÁTICA • INGENIERÍA DE SOFTWARE

REPORT / INFORME TÉCNICO DE SOFTWARE

Laboratorio Canary

Estrategia de despliegue Canary.

ESTUDIANTE Gerardo Abraham Barrón Gómez	**EXPERIENCIA EDUCATIVA** Despliegue de software
DOCENTE / CATEDRÁTICO Saúl Isidro Domínguez	**FECHA DE ENTREGA** 14 de Octubre de 2026

XALAPA, ENRÍQUEZ, VERACRUZ • MÉXICO

Objetivo:

Aplicar la estrategia de despliegue Canary con el repositorio proporcionado por el profesor.

Preparación en la VM Vagrant:

```
 => => naming to docker.io/library/canary-lab:v2                                                    0.0s
 => => unpacking to docker.io/library/canary-lab:v2                                                 0.0s
 => [blue] resolving provenance for metadata file                                                   0.0s
 => [green] resolving provenance for metadata file                                                  0.0s
[+] up 6/6
 ✔Image canary-lab:v1          Built                                                               2.1s
 ✔Image canary-lab:v2          Built                                                               2.0s
 ✔Network canary_lab_canary    Created                                                             0.1s
 ✔Container canary_lab-green-1 Started                                                             0.9s
 ✔Container canary_lab-blue-1  Started                                                             0.6s
 ✔Container canary_lab-proxy-1 Started                                                             1.1s
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab$ docker compose ls
NAME                STATUS              CONFIG FILES
apihub              running(1)          /vagrant_data/apiHub/docker-compose.yml
canary_lab          running(3)          /vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/compose
.yaml
customer-service    running(1)          /vagrant_data/fia-corp-pos/server/customer-service/docker-compose.yml
server              running(3)          /vagrant_data/fia-corp-pos/server/docker-compose.yml
stackdb             running(3)          /vagrant/stackdb/stackdb.yml
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab$ docker compose ps
NAME                 IMAGE              COMMAND                  SERVICE   CREATED          STATUS          PORTS
canary_lab-blue-1    canary-lab:v1      "docker-entrypoint.s…"   blue      28 seconds ago   Up 27 seconds   3000/tcp
canary_lab-green-1   canary-lab:v2      "docker-entrypoint.s…"   green     28 seconds ago   Up 27 seconds   3000/tcp
canary_lab-proxy-1   nginx:1.27-alpine   "/docker-entrypoint.…"   proxy     28 seconds ago   Up 27 seconds   0.0.0.0:809
0->80/tcp, [::]:8090->80/tcp
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab$ curl -s http://localhost:8090
/api/info
{"service":"canary-demo","version":"v1"}vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/c
```

Etapa 1: línea base (Blue 100%)

Registro de distribución y verificación de que Green existe pero no recibe tráfico público.

compose.yaml | active.conf

deployment-strategies > demo_canary > canary_lab > nginx > active.conf

```
upstream backend {
    server blue:3000;
}
server {
    listen 80;
    access_log /var/log/nginx/access.log;
    location / {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header Connection "close";
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Ilustración 1 nginx/active.conf

```
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab$ cd scripts
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./promote.sh blue
/usr/bin/env: ‘bash\r’: No such file or directory
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ sed -i 's/\r$//' promote.sh
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./promote.sh blue
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/10/15 20:07:31 [notice] 29#29: signal process started
Etapa activada: blue
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$
```

```
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./sample.sh 100
/usr/bin/env: ‘bash\r’: No such file or directory
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ sed -i 's/\r$//' sample.sh
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./sample.sh 100
    100 "version":"v1"
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$
```

Ejecutamos los primeros scripts. Y revisamos los contenedores que tenemos dentro de nuestro proyecto.

```
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ docker compose ps
NAME                IMAGE               COMMAND                  SERVICE   CREATED          STATUS          PORTS
canary_lab-blue-1   canary-lab:v1       "docker-entrypoint.s…"   blue      27 minutes ago   Up 27 minutes   3000/tcp
canary_lab-green-1  canary-lab:v2       "docker-entrypoint.s…"   green     27 minutes ago   Up 27 minutes   3000/tcp
canary_lab-proxy-1  nginx:1.27-alpine   "/docker-entrypoint.…"   proxy     27 minutes ago   Up 27 minutes   0.0.0.0:8090->80/tcp, [::]
:8090->80/tcp
```

Etapa 2: Canary al 10%

```
{"service":"canary-demo","version":"v1"}vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scrip
ts$ ./promote.sh canary10
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/10/15 20:22:38 [notice] 43#43: signal process started
Etapa activada: canary10
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./sample.sh 100
     90 "version":"v1"
     10 "version":"v2"
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$
```

¿Por qué el peso 9:1 no equivale a una probabilidad garantizada para cada grupo de diez solicitudes ni a 10% de usuarios únicos?

Porque nginx redirige el tráfico de las solicitudes a la nueva versión pero las maneja a nivel general, no por usuario, por eso es que no llega a 10% de los usuarios, llega al 10% de las solicitudes.

El peso 9:1 np es una probabilidad garantizada porque su razonamiento está basado en probabilidad estadística, puede que reciba 10 solicitudes, pero por probabilidad puede que no necesariamente 1 de esas solicitudes sea redirigida. En grandes cantidades las cantidades si llegan a aproximarse.

```
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ curl -s http://127.0.0.1:8090/api/in
fo
{"service":"canary-demo","version":"v1"}vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scrip
```

Aquí hacemos una petición pero por estadística hemos sido atendidos en la v1.

Etapa 3: Promoción gradual

```
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./promote.sh canary50
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/10/15 20:30:37 [notice] 57#57: signal process started
Etapa activada: canary50
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./sample.sh 100
     51 "version":"v1"
     49 "version":"v2"
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./promote.sh green
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/10/15 20:31:20 [notice] 71#71: signal process started
Etapa activada: green
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./sample.sh 30
     30 "version":"v2"
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$
```

Explicar qué cambia entre una liberación canry y una blue-green:

La liberación en blue-green es abrupta, inmediatamente se redirigen todas las peticiones(100 %) a la nueva versión, mientras que en canary el despliegue es progresivo, primero se redirige un pequeño porcentaje del total de las solicitudes y si toda va bien y se cumplen con los criterios acordados entonces se aumenta la cantidad de solicitudes redirigidas hasta llegar al 100%.

¿Qué métricas consultarías antes de pasar del 10% al 50%?

Pienso que en este punto como hablamos desde un contexto de redes, lo que más importa es el rendimiento métricas como la latencia, si hay reintentos en las peticiones, códigos de error, estado de nuestros servidores y sus recursos.

```
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ curl -s http://127.0.0.1:8090/api/in
fo
{"service":"canary-demo","version":"v2"}vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scrip
ts$
```

Aquí hemos vuelto a consultar y ahora vemos que ya fuimos redirigidos a la versión 2.

Etapa 4: Inyección de fallos y roll back

```
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./promote.sh blue
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/10/15 20:39:01 [notice] 85#85: signal process started
Etapa activada: blue
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ GREEN_FAIL=true docker compose up -d
 --force-recreate green
[+] up 1/1
 ✔Container canary_lab-green-1 Started                                                                                    10.6s
```

```
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./promote.sh canary10
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/10/15 20:41:52 [notice] 100#100: signal process started
Etapa activada: canary10
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./check-checkout.sh 50
/usr/bin/env: ‘bash\r’: No such file or directory
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ sed -i 's/\r$//' check-checkout.sh
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./check-checkout.sh 50
     45 200
      5 503
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$
```

```
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./promote.sh blue
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/10/15 20:46:54 [notice] 114#114: signal process started
Etapa activada: blue
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ ./check-checkout.sh 20
     20 200
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ GREEN_FAIL=false docker compose up -
d --force-recreate green
[+] up 1/1
 ✔Container canary_lab-green-1 Started                                                                                          10.7s
vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scripts$ curl -s http://127.0.0.1:8090/api/in
fo
{"service":"canary-demo","version":"v1"}vagrant@revproxy:/vagrant_data/deploy-canary/deployment-strategies/demo_canary/canary_lab/scrip
ts$
```

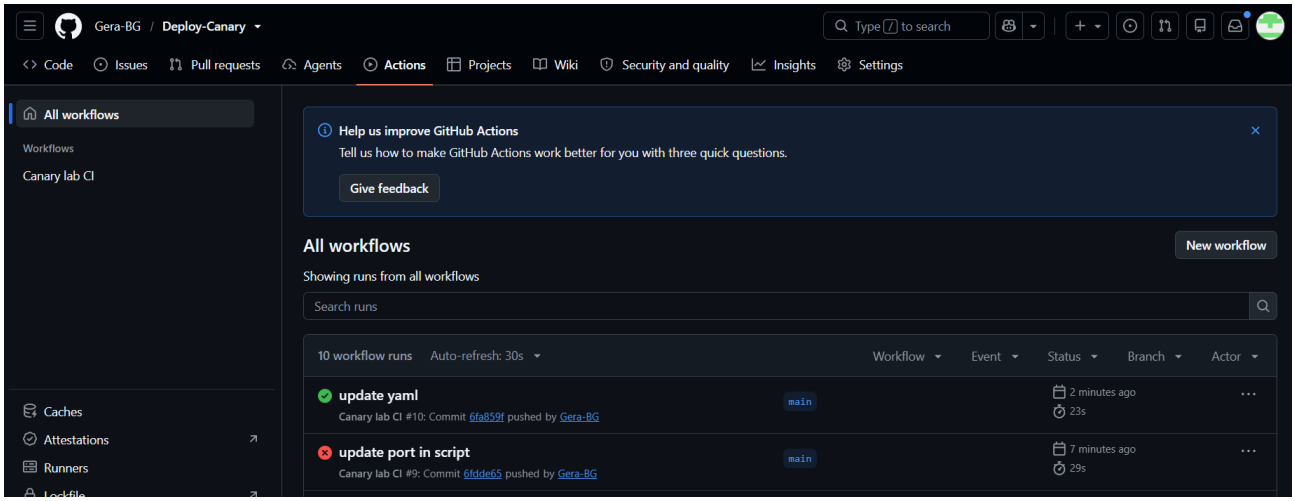
¿Porqué el health check no detectó el problema? ¿Qué consecuencias tendría mantener la promoción? ¿Qué pasaría si una migración de base de datos de v2 fuese incompatible con v1?

Porque health solo verifica que el contenedor está en ejecución y el servidor web puede recibir peticiones. Los fallos internos que afectan el procesamiento de un proceso y un flujo de negocio específico o la lógica de una ruta particular no son detectables, por lo que el servidor sigue respondiendo que está "sano" en su ruta de estado. Mantener la promoción significa aumentar gradualmente el tráfico hacia la versión Green. Si se ignora que la versión Green está devolviendo respuestas 503 y se dirige el 100% del tráfico hacia ella, el fallo que solo afectaba a un pequeño porcentaje de usuarios provocará una interrupción total del servicio en producción. Con Canary, ambas versiones (v1 y v2) funcionan al mismo tiempo y, por lo general, apuntan a la misma base de datos. Si la v2 aplica una migración incompatible la versión v1 que aún atiende a la mayoría del tráfico comenzaría a fallar masivamente porque su código espera el esquema antiguo. En consecuencia, esto hace imposible hacer un *rollback*, ya que el código de la v1 seguiría siendo incompatible con el nuevo estado de la base de datos.

Github Actions

```yaml
name: Canary lab CI
on:
  push:
  pull_request:
  workflow_dispatch:
jobs:
  test-and-canary:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - name: Unit tests
        run: node --test test/*.test.js
      - name: Start two releases and proxy
        run: docker compose up -d --build
      - name: Wait for HTTP readiness
        run: |
          for i in $(seq 1 30); do
            if curl -fsS http://127.0.0.1:8080/health; then exit 0; fi
            sleep 2
          done
          docker compose logs
          exit 1
      - name: Validate canary routing
        run: |
          ./scripts/promote.sh canary10
          ./scripts/sample.sh 100 | tee /tmp/distribution.txt
          grep -q '"version":"v1"' /tmp/distribution.txt
          grep -q '"version":"v2"' /tmp/distribution.txt
      - name: Promote and rollback
        run: |
          ./scripts/promote.sh green
          curl -fsS http://127.0.0.1:8080/api/info | grep -q '"version":"v2"'
          ./scripts/promote.sh blue
```

```
      grep -q '"version":"v1"' /tmp/distribution.txt
      grep -q '"version":"v2"' /tmp/distribution.txt
  - name: Promote and rollback
    run: |
      ./scripts/promote.sh green
      curl -fsS http://127.0.0.1:8080/api/info | grep -q '"version":"v2"'
      ./scripts/promote.sh blue
      curl -fsS http://127.0.0.1:8080/api/info | grep -q '"version":"v1"'
  - name: Logs on failure
    if: failure()
    run: docker compose logs
  - name: Tear down
    if: always()
    run: docker compose down -v
```



El archivo YAML de GitHub Actions se compone de bloques principales que definen cuándo, dónde y cómo se ejecutan las tareas: tenemos name donde definimos el nombre descriptivo del action que aparecerá en la pestaña Actions de GitHub. Luego tenemos on, que especifica los eventos que activan el flujo de trabajo (push, pull_request o workflow_dispatch). Después están los Jobs que contiene el conjunto de trabajos que se ejecutarán. Tenemos runs-on que define el entorno o la máquina virtual donde se correrán los comandos (en este caso, ubuntu-latest). steps es la lista secuencial de pasos que ejecuta el trabajo.

En un despliegue de tipo Canary, donde la versión nueva convive con la versión actual antes de recibir el 100% del tráfico, existen ciertas validaciones puede causar caídas masivas en producción.

Pruebas unitarias e integración previa: Debemos garantizar que la nueva versión compila, pasa sus tests unitarios y cumple con la sintaxis antes de generar la imagen de Docker o iniciar contenedores.

Verificación de salud profunda (*Deep Health Check*) Porque el endpoint de salud debe validar conexiones a la base de datos, dependencias de red y otros servicios. Mecanismo automático de Rollback (Reversión): Si

cualquier prueba de integración o muestreo falla en la fase Canary, el sistema debe ser capaz de redirigir instantáneamente el 100% del tráfico de vuelta a la versión estable antes de que afecte a más usuarios.

Preguntas

1. ¿Qué diferencia existe entre distribuir el 10 % de solicitudes y liberar a un 10 % fijo de usuarios?

Redirigir el 10 porciento de las solicitudes significa que sin importar el usuario el proxy se enfoca en las peticiones, y liberar el 10% fijo de usuarios implica que sin importar cuantas solicitudes sean, nos enfocamos en los usuarios.

2. ¿Por qué una respuesta 200 en /health no basta para aprobar una versión?

Porque hay muchas implicaciones, monitorear y revisar el estado de las versiones. Health solo implica que el servidor está encendido y responde pero no dice más allá sobre si procesa correctamente las solicitudes.

3. ¿Qué aporta Canary frente a Blue-Green y qué costo adicional introduce?

Canary aporta una transición más suave y progresiva que blue-green sin embargo, requiere más costos y estar pendiente de que cada porcentaje en aumento sigue cumpliendo con los criterios de aceptación hasta llegar al 100 %.

4. ¿Qué estrategia aplicarías si la versión nueva modifica de forma irreversible el esquema de datos?

Pues lo ideal sería prevenirlo haciendo que el esquema de base de datos de ambas versiones sea compatible, así no habría problemas en la transición. SI hay que modificar columnas antes del despliegue las agrego y hago el despliegue, si veo que todo funciona bien y ya llegué al 100 % entonces ahora elimino aquellos elementos que están de sobra.

5. ¿Por qué GitHub Actions puede validar Compose pero no necesariamente desplegar en tu VM Vagrant?

GitHub Actions ejecuta el workflow en un runner (su propia máquina virtual de GitHub). En ese runner: Se clona tu repositorio. Se instala Docker y Docker Compose. Se ejecuta docker compose up -d –build y se validan los servicios. El runner tiene su propio Docker, su propia red, y sus propios contenedores. Es un entorno perfecto para probar que funciona. Cuenta con ya que corre en los servidores de Microsoft (Azure), no en mi computadora a nivel local. Si fuera en mi VM habría un conflicto con las Credenciales y secretos ya que sería necesario exponer la VM a Internet, usar SSH, configurar claves SSH como secretos en GitHub, etc.